Informe de resultados de tratamiento de imagen

Estudiante: Jerson Baquero Cháves | **Universidad de Cundinamarca** | Curso CADI Percepción Computacional – Semana 8

1. Introducción y objetivo

Este informe analiza cómo las condiciones de captura (iluminación, movimiento, fondo y perspectiva) afectan la calidad de una imagen y, por tanto, su utilidad en un sistema de percepción computacional. Se fotografió un mismo objeto (un frasco con tapa verde) en cinco condiciones, se diagnosticaron con Python y OpenCV en Google Colab, y se eligió la imagen más problemática para aplicar un tratamiento justificado y evaluarlo con evidencia visual y numérica.

2. Descripción de las imágenes y condiciones de captura



Figura 1. Las cinco capturas del mismo objeto.

1_referencia: toma frontal con luz directa intensa; **2_movimiento:** captura con movimiento de la cámara; **3_angulo:** vista inclinada y más cercana del frasco; **4_fondo_complejo:** vista cenital sobre una superficie de madera con veta y sombras marcadas; **5_baja_luz:** captura casi sin iluminación. Todas son JPEG de 739x1600 px (WhatsApp recomprime y redimensiona las fotos, lo que homogeniza la resolución y condiciona el análisis).

3. Diagnóstico y tabla comparativa

Con OpenCV se calcularon: resolución y canales (`img.shape`), brillo (media de la escala de grises), contraste (desviación estándar), nitidez (varianza del Laplaciano, sobre toda la imagen y sobre una región de interés del objeto), porcentaje de píxeles oscuros (<50) y formato/tamaño del archivo.

Imagen	Resol.	Can.	Brillo	Contr.	Nitidez (global)	Nitidez (ROI)	% oscuros	Tam. KB
1_referencia	739x1600	3	216.7	44.3	10.8	40.2	0.0	61.7
2_movimiento	739x1600	3	95.2	35.3	9.1	14.7	11.2	66.2
3_angulo	739x1600	3	104.6	44.5	418.0	505.6	10.9	184.2
4_fondo_complejo	739x1600	3	94.0	49.6	453.8	175.0	24.5	196.7
5_baja_luz	739x1600	3	7.8	6.5	0.3	1.5	100.0	15.0

Tabla 1. Métricas por imagen (la fila resaltada es la imagen elegida).

Principales problemas encontrados. (a) La imagen de *baja luz* es la más crítica: brillo medio de 7,8/255, contraste de 6,5 y 100 % de píxeles oscuros, por lo que el objeto casi no se distingue. (b) La imagen de *movimiento* presenta la menor nitidez sobre el objeto (14,7 frente a 40,2 de la referencia). (c) La *referencia* resultó sobreexpuesta (brillo 216,7) por la luz directa, con nitidez baja en el objeto; no es una referencia ideal y se reconoce como limitación. (d) En *ángulo* y *fondo complejo* la nitidez global es alta (418 y 454), pero engañosa: la veta de la madera aporta bordes que inflan el Laplaciano, por eso se midió también sobre el objeto. (e) Los archivos pesan menos cuando hay poca información: 15,0 KB en baja luz frente a 184-197 KB en las imágenes con textura.

4. Problema seleccionado y diseño de la solución

Se seleccionó **5_baja_luz** porque es la imagen donde la información visual se pierde casi por completo; cualquier tarea posterior (segmentación, detección de bordes, reconocimiento) fallaría. Los problemas son brillo y contraste muy bajos, por lo que el tratamiento debe actuar sobre la **luminancia** y no desenfocar ni alterar la forma. Se diseñó un proceso de dos pasos:
1) Corrección gamma (γ = 0,45) mediante una tabla de búsqueda (LUT): eleva de forma no lineal los tonos oscuros sin saturar los claros. **2) CLAHE** (clipLimit = 3,0, mosaico 8x8) aplicado solo al canal L del espacio LAB: aumenta el contraste local sin distorsionar el color y limita la amplificación del ruido. Se descartó la ecualización global del histograma porque en una imagen tan oscura amplifica el ruido de forma uniforme y produce un aspecto artificial.

Fragmentos relevantes del script

```
def metricas(img, ruta):
    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return {"brillo_medio": gris.mean(), "contraste_std": gris.std(),
            "nitidez_lap_var": cv2.Laplacian(gris, cv2.CV_64F).var(),
            "pct_pix_oscuros(<50)": (gris < 50).mean()*100,
            "tam_KB": os.path.getsize(ruta)/1024}

def tratar(img, gamma=0.45, clip=3.0):
    lut = np.array([((i/255.0)**gamma)*255 for i in range(256)]).astype("uint8")
    g = cv2.LUT(img, lut)                                    # 1) gamma
    lab = cv2.cvtColor(g, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    l = cv2.createCLAHE(clipLimit=clip, tileGridSize=(8, 8)).apply(l)  # 2) CLAHE en L
    return cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)
```

El script completo recorre las cinco imágenes, calcula las métricas, guarda la tabla en CSV, aplica `tratar()` a la imagen de baja luz y recalcula las métricas para la comparación.

5. Comparación antes / después

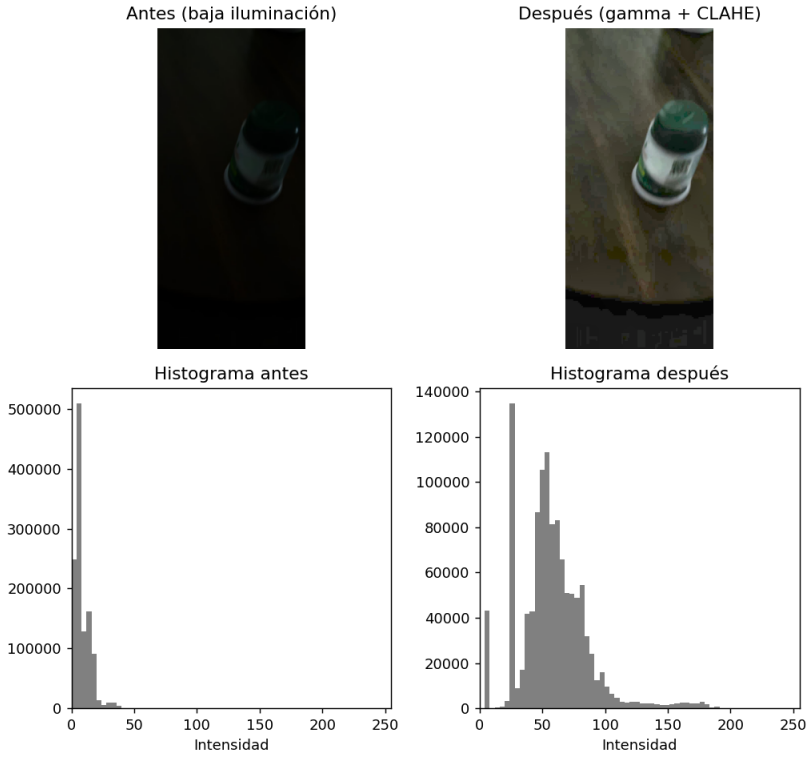


Figura 2. Imagen original y tratada, con sus histogramas de intensidad.

Métrica	Antes	Después
brillo_medio	7.8	58.2
contraste_std	6.5	27.8
nitidez_lap_var	0.3	15.9
pct_pix_oscuros(<50)	100.0	36.2
nitidez_ROI	1.5	42.0
ruido_est(std de g - blur)	0.17	1.17

Tabla 2. Resultados numéricos antes y después del tratamiento.

6. Análisis de resultados y limitaciones

El tratamiento produjo una mejora real y medible: el brillo medio pasó de 7,8 a 58,2, el contraste de 6,5 a 27,8, los píxeles oscuros bajaron de 100 % a 36,2 % y la nitidez sobre el objeto subió de 1,5 a 42,0. Visualmente el frasco, la tapa, el código QR y la veta de la mesa vuelven a distinguirse, y el histograma, antes concentrado en 0-30, ahora se extiende hasta cerca de 190.

Sin embargo, hay limitaciones que conviene reconocer. (1) **El ruido aumentó** (estimación de 0,17 a 1,17), porque el realce amplifica también el ruido del sensor. (2) La **nitidez medida sube en parte por ese ruido**: el Laplaciano no distingue entre detalle real y ruido, así que no toda la mejora de 1,5 a 42,0 es información recuperada. (3) Aparece **posterización o bandas** en las zonas más oscuras (parte inferior), ya que la imagen original, comprimida en JPEG y con apenas 15 KB, casi no contiene información allí. (4) El tratamiento **no crea información** que no se capturó: solo la hace visible. La referencia sobreexpuesta y el desenfoque por movimiento no se resuelven con este método; el desenfoque requeriría técnicas distintas (por ejemplo, deconvolución) y tiene límites similares.

7. Conclusiones y recomendaciones

El diagnóstico previo con métricas permitió elegir un tratamiento coherente con el problema (luminancia insuficiente) en lugar de aplicar filtros al azar. Gamma más CLAHE en el canal L mejoró notablemente la legibilidad del objeto, aunque a costa de más ruido. Se recomienda: capturar con iluminación suave y difusa para evitar tanto la penumbra como la sobreexposición; usar fondos uniformes cuando se busque segmentar el objeto; sujetar el dispositivo o usar un apoyo para evitar el movimiento; y, al medir nitidez, hacerlo sobre el objeto y no sobre toda la imagen. Como trabajo futuro, un filtro de reducción de ruido (por ejemplo `cv2.fastNlMeansDenoisingColored`) antes del realce podría reducir el ruido amplificado.

8. Enlace al código

Repositorio en GitHub: https://github.com/Jerson09/tratamientoImg.git
Notebook de Colab: https://colab.research.google.com/drive/149pIfmBcKzk4Fi6a6zAxDU17ot-6cw3n